LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK

Identitas	
Nama	M.Fahmi Hadi Wijaya
NIM	251511016
Kelas	2A-D3
Repository GitHub	https://github.com/fhxwjxy/MFahmi-Hadi-Wijaya-251511016-PBO

Bagian A - Circle dan Cylinder

Bagian ini membangun hubungan Circle -> Cylinder secara bertahap dengan alur PREDICT -> CODE -> RUN -> OBSERVE -> EXPLAIN.

A.1 Step 1 - Circle versi paling sederhana

```
c1
radius = 1.0
area   = 3.141592653589793
c2
radius = 2.5
area   = 19.634954084936208
Circle[radius=2.5, color=red]
```

Gambar 1. Output step 1

Checkpoint 1: Mana state Circle? Mana behavior Circle? Apa perbedaan state dua object yang dibuat?

Jawaban: State: radius dan color. Behavior: getRadius(), getArea(), toString(). Object pertama radius 1.0 dan object kedua radius 2.5, sedangkan color sama-sama red. Keduanya punya salinan state sendiri-sendiri.

A.2 Step 2 - Melengkapi encapsulation Circle

```
Circle[radius=1.0, color=red]
Circle[radius=2.0, color=yellow]
Circle[radius=3.0, color=blue]
```

Gambar 2. Output step 2

Checkpoint 2: Mengapa code di luar Circle tidak mengubah radius secara langsung? Apa peran method public terhadap state private?

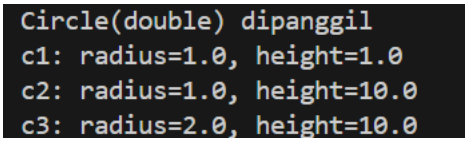
Jawaban: Field radius private sehingga hanya bisa diakses di dalam Circle. Method public (getter/setter) menjadi pintu akses yang terkontrol oleh class pemiliknya.

A.3 Step 3 - Prediksi sebelum membuat Cylinder

Prediksi ditulis sebelum coding.

Pertanyaan	Prediksi saya
Apakah radius dan color perlu ditulis ulang sebagai field Cylinder?	Tidak perlu, karena Cylinder dibuat sebagai subclass, jadi radius dan color sudah dimiliki lewat circle.
Behavior apa dari Circle yang masih berguna untuk Cylinder?	getRadius(), setRadius(), getColor(), setColor(), dan getArea(). Alas silinder berbentuk lingkaran, jadi getArea() bisa dipakai untuk menghitung volume (luas alas × tinggi).
State apa yang hanya dimiliki Cylinder?	height (tinggi). Itu satu-satunya state tambahan yang tidak ada di Circle.

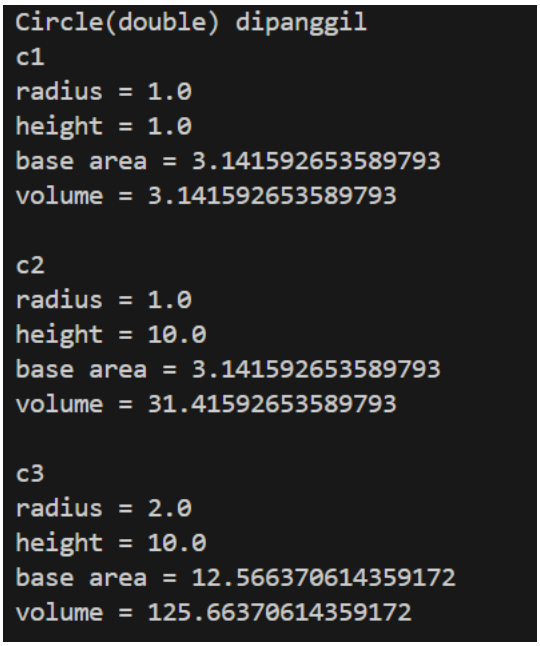
A.4 Step 4 - Cylinder sebagai subclass Circle



Gambar 3. Output Step 4.

Kesimpulan: Prediksi saya benar. radius dan color tidak perlu ditulis ulang di Cylinder karena diwarisi lewat extends Circle. getArea() terbukti berguna sebagai luas alas pada getVolume(), dan satu-satunya state tambahan Cylinder memang height.

A.5 Step 5 - Uji inherited method



Gambar 4. Output Step 5.

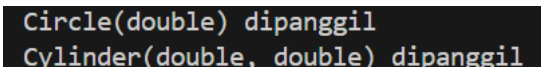
Method yang ditulis di Cylinder dibandingkan dengan method yang hanya diwarisi dari Circle:

Ditulis di Cylinder.java	**Diwarisi dari Circle (dapat dipanggil, tidak ditulis di Cylinder)**
getHeight(), setHeight(), getVolume()	getRadius(), setRadius(), getColor(), setColor(), getArea() (sebelum di-override)

Checkpoint 3: Mengapa c3.getRadius() dan c3.getArea() dapat dipanggil padahal tidak ditulis ulang di Cylinder?

Jawaban: Karena Cylinder extends Circle. Method public milik Circle ikut tersedia pada object Cylinder.

A.6 Step 6 - Constructor chain dan fungsi super(...)



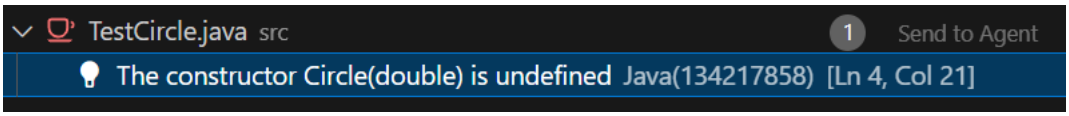
Gambar 5. Output step 6.

Eksperimen A - super(radius) dijadikan komentar

```
Cylinder(double, double) dipanggil
```

Gambar 6. Output ketika super(radius); dihilangkan sementara

Eksperimen B - error compile pada constructor



Gambar 7. Output Ketika test eror yaitu dengan menghilangkan kode circle(double radius) sementara

Checkpoint 4: Apa yang dikerjakan lebih dahulu saat object Cylinder dibuat: bagian Circle atau Cylinder? Bukti apa dari output?

Jawaban: Bagian Circle. Output "Circle(double) dipanggil" muncul sebelum "Cylinder(double,double) dipanggil".

A.7 Step 7 - Batas encapsulation pada subclass



Gambar 8. Output ketika menulis dengan return radius;

```
Circle(double) dipanggil
Cylinder(double, double) dipanggil
c1: radius=1.0, height=1.0
c2: radius=1.0, height=10.0
c3: radius=2.0, height=10.0
```

Gambar 9. Output ketika mengganti return radius dengan return getRadius();

Checkpoint 5: Extends membuat subclass mewarisi behavior yang dapat diakses, tetapi tidak membuat field private superclass menjadi ...

Jawaban: ... dapat diakses langsung oleh subclass. Akses harus lewat method public/protected milik superclass, misalnya getRadius().

A.8 Step 8 - Override getArea() dan bug pada getVolume()

Tahap 1 - Sebelum override

```
Circle(double) dipanggil
Cylinder(double, double) dipanggil
c1
radius = 1.0
height = 1.0
base area = 3.141592653589793
volume = 3.141592653589793

c2
radius = 1.0
height = 10.0
base area = 3.141592653589793
volume = 31.41592653589793

c3
radius = 2.0
height = 10.0
base area = 12.566370614359172
volume = 125.66370614359172
```

Gambar 10. Output TestCylinder sebelum getArea() di-override.

Tahap 2 - Setelah override, getVolume() masih memakai getArea()

```
Circle(double) dipanggil
Cylinder(double, double) dipanggil
c1
radius = 1.0
height = 1.0
base area = 12.566370614359172
volume = 3.141592653589793

c2
radius = 1.0
height = 10.0
base area = 69.11503837897544
volume = 31.41592653589793

c3
radius = 2.0
height = 10.0
base area = 150.79644737231007
volume = 125.66370614359172
```

Gambar 11. Output kode override

Setelah getArea() di-override menjadi luas permukaan silinder, nilainya berubah dari 12.566 menjadi 150.796. Kode getVolume() tidak diubah, tetapi volume menjadi 1507.96, yaitu 10 kali lebih besar dari seharusnya. Hasil ini salah karena getVolume() ikut memakai luas permukaan, bukan luas alas.

Checkpoint 6: Mengapa getVolume() berubah menjadi salah dan apa fungsi super.getArea()?

Jawaban: getVolume() memanggil getArea(); karena objeknya Cylinder, versi override (luas permukaan) yang dijalankan. super.getArea() secara eksplisit meminta implementasi Circle (luas alas).

A.9 Step 9 - Override toString() dan uji @Override

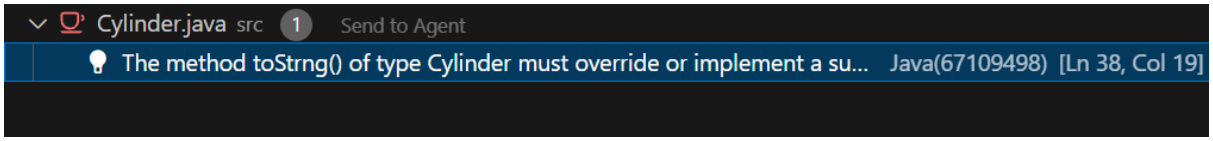
```
Circle(double) dipanggil
Cylinder(double, double) dipanggil
c1
radius = 1.0
height = 1.0
base area = 12.566370614359172
volume = 12.566370614359172

c2
radius = 1.0
height = 10.0
base area = 69.11503837897544
volume = 691.1503837897544

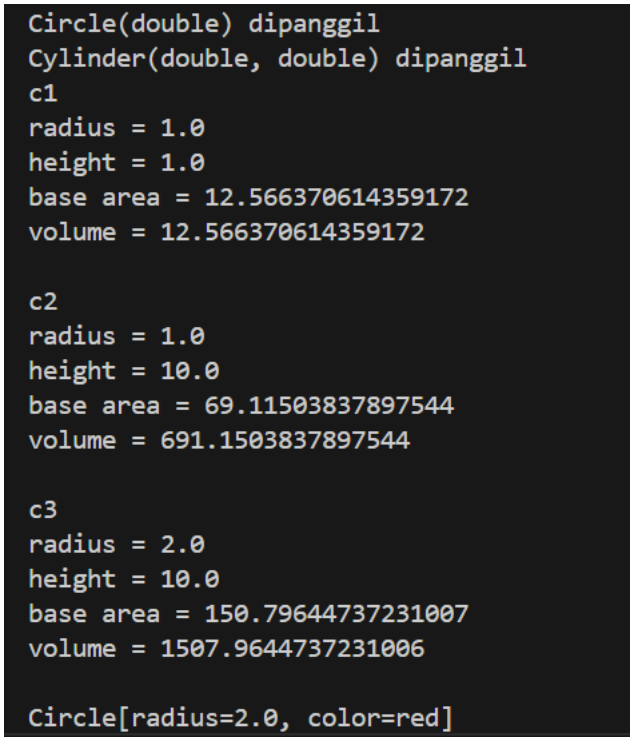
c3
radius = 2.0
height = 10.0
base area = 150.79644737231007
volume = 1507.9644737231006

Cylinder: subclass of Circle[radius=2.0, color=red], height=10.0
```

Gambar 12. Output menjadikan toString() menjadi override



Gambar 13. Error compile saat method diganti nama menjadi toStrng() dengan @Override tetap ada.



Gambar 14. Setelah @Override dihapus (nama masih toStrng()): compile berhasil.

Checkpoint 7: @Override tidak membuat inheritance terjadi. Lalu apa kegunaannya?

Jawaban: @Override adalah pemeriksaan compiler: memastikan method memang menimpa method superclass sehingga typo nama atau signature langsung menjadi error.

Bagian B - Shape Hierarchy

Bagian ini menarik kemiripan Circle, Rectangle, dan Square ke superclass Shape, lalu mempertahankan perbedaannya di subclass.

B.1 Step 10 - Superclass Shape

```
Shape[color=green, filled=true]
Shape[color=black, filled=false]
```

Gambar 15. Kode Shape.java.

Checkpoint 8: State apa yang cukup umum sehingga masuk akal dimiliki Circle maupun Rectangle?

Jawaban: color dan filled

B.2 Step 11 - Circle sebagai specialization dari Shape

```
Circle[radius=1.0, Shape[color=green, filled=true]]
area=3.141592653589793, perimeter=6.283185307179586, color=green, filled=true
Circle[radius=2.5, Shape[color=green, filled=true]]
area=19.634954084936208, perimeter=15.707963267948966, color=green, filled=true
Circle[radius=3.0, Shape[color=blue, filled=false]]
area=28.274333882308138, perimeter=18.84955592153876, color=blue, filled=false
```

Gambar 16. Kode Circle.java versi extends Shape.

Daftar behavior (Checkpoint 9):

Berasal dari Shape	Spesifik milik Circle
getColor(), setColor(), isFilled(), setFilled()	getRadius(), setRadius(), getArea(), getPerimeter()

B.3 Step 12 - Rectangle sebagai specialization dari Shape

```
Rectangle[width=1.0, length=1.0, Shape[color=green, filled=true]]
area=1.0, perimeter=4.0
Rectangle[width=3.0, length=4.0, Shape[color=green, filled=true]]
area=12.0, perimeter=14.0
Rectangle[width=3.0, length=4.0, Shape[color=yellow, filled=true]]
area=12.0, perimeter=14.0
```

Gambar 17. Kode Rectangle.java.

Checkpoint 10: Apa kemiripan Circle dan Rectangle yang sudah ditarik ke Shape? Apa yang sengaja tetap berbeda?

Jawaban: Yang ditarik ke Shape: color, filled, dan getter/setter-nya. Yang tetap berbeda: field ukuran (radius vs width/length) dan rumus getArea()/getPerimeter().

B.4 Step 13 - Square dari Rectangle

B.5 Step 14 - Pelanggaran aturan Square (sebelum override)

```
awal : Square[side=5.0, Rectangle[width=5.0, length=5.0, Shape[color=green, filled=true]]]
area : 25.0
sesudah setWidth(8): Square[side=8.0, Rectangle[width=8.0, length=5.0, Shape[color=green, fil
led=true]]]
area : 40.0
width=8.0, length=5.0
```

Gambar 18. Output setelah s.setWidth(8.0) pada Square sebelum setter di-override: width=8.0, length=5.0.

Checkpoint 11: Mengapa behavior warisan yang tersedia secara teknis tidak selalu sesuai aturan subclass?

Jawaban: Setter warisan dari Rectangle mengubah satu sisi saja, sehingga Square bisa memiliki width dan length berbeda. Override diperlukan untuk menjaga aturan Square.

B.6 Step 15 - Override setter agar Square tetap valid

```
awal : Square[side=5.0, Rectangle[width=5.0, length=5.0, Shape[color=green, filled=true]]]
area : 25.0
sesudah setWidth(8): Square[side=8.0, Rectangle[width=8.0, length=8.0, Shape[color=green,
filled=true]]]
area : 64.0
width=8.0, length=8.0
```

Gambar 19. Kode setWidth() dan setLength() yang di-override di Square.

Checkpoint 12: Mengapa setWidth()/setLength() perlu dioverride, tetapi getArea()/getPerimeter() tidak?

Jawaban: Setter bisa merusak aturan Square sehingga harus dioverride. Rumus width*length dan 2*(width+length) tetap benar selama width = length, jadi warisan dari Rectangle cukup (dibuktikan dari output).

B.7 Step 16 - Test hierarchy secara utuh

```
Shape[color=black, filled=false]
Circle[radius=2.0, Shape[color=blue, filled=true]] area=12.566370614359172
Rectangle[width=3.0, length=4.0, Shape[color=yellow, filled=true]] area=12.0
Square[side=5.0, Rectangle[width=5.0, length=5.0, Shape[color=green, filled=true]]] area=2
5.0
```

Gambar 20. Output akhir TestShape.

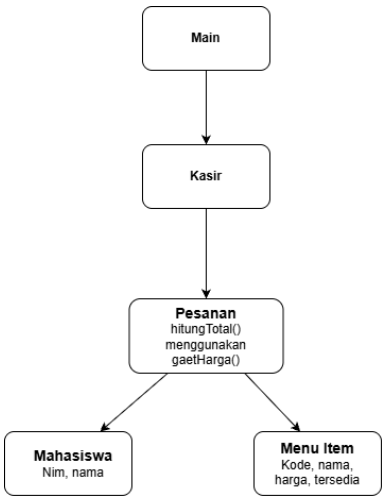
Bagian C - Kasus Tugas Sebelumnya (Kasir Kantin)

Kasus: aplikasi pemesanan kantin dengan class Mahasiswa, MenuItem, Pesanan, Kasir, dan Main. Inheritance diterapkan pada MenuItem menjadi Makanan dan Minuman.

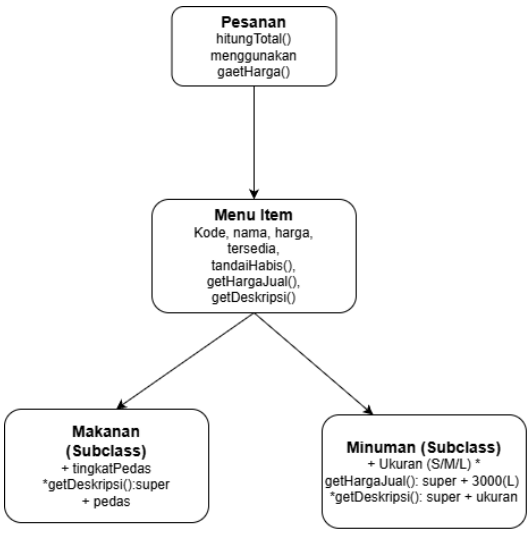
C.1 Step 17 - Kandidat generalization

	Makanan	**Minuman**
State/behavior sama (di MenuItem)	kode, nama, harga, tersedia, tandaiHabis()	kode, nama, harga, tersedia, tandaiHabis()
Perbedaan	tingkatPedas	ukuran (S/M/L), harga tambahan untuk ukuran L
Kalimat uji	"Makanan adalah sebuah MenuItem" (masuk akal)	"Minuman adalah sebuah MenuItem" (masuk akal)

C.2 Step 18 - Diagram sebelum coding



Gambar 21. Diagram hierarchy SEBELUM refactor (Main - Kasir - Pesanan - Mahasiswa/MenuItem).



Gambar 22. Diagram hierarchy SESUDAH refactor. Tandai superclass, subclass, state/behavior umum dan spesifik, serta method yang di-override.

C.3 Step 19 – Implementasi

```
=== Demo inheritance ===
[M01] Nasi Goreng (pedas level 3) | harga jual 18000
[M02] Kopi Susu (ukuran M) | harga jual 12000
[M03] Es Teh (ukuran L) | harga jual 8000
Pesanan #4 - Siti memesan Es Teh x2
Total P4: 16000
Status P4 setelah menu habis: Tidak dapat diproses
Pesanan Ditolak
Jumlah pesanan dibuat: 4
```

Gambar 23. Kode MenuItem.java (tambahan getHargaJual() dan getDeskripsi()).

Checkpoint dan Refleksi

D.1 Definition of Done

Butir	Status
Semua Step 1-16 telah dicoba, bukan hanya code final disalin	ya
Output dan minimal satu error eksperimen tersimpan sebagai bukti	ya
Dapat menunjukkan method yang diwarisi dan method yang dioverride	ya
Dapat menjelaskan urutan constructor dan fungsi super(...)	ya
Dapat menjelaskan mengapa super.getArea() diperlukan pada getVolume()	ya
Dapat menjelaskan mengapa Square perlu override setter	ya
Kasus tugas sebelumnya direfactor dan lulus skenario uji utama	ya
PDF dan source siap di GitHub	ya

D.2 Self Assessment

Pernyataan	Skor 1-5
Saya dapat menjelaskan extends dari code yang saya jalankan.	3
Saya dapat membedakan inherited method dan overridden method.	2
Saya memahami mengapa constructor subclass memanggil super(...).	2
Saya dapat menggunakan super.method(...) dengan alasan yang tepat.	1
Saya dapat menjaga encapsulation pada hierarchy class.	2
Saya dapat menentukan kapan inheritance masuk akal pada kasus sendiri.	2